

LLM-Based Semantic Chunking for Retrieval-Augmented Generation

Compared with rule-based and embedding-based splitting from LangChain

Ivan Antunovic

Supervisor: Dr. Marian Lux · University of Vienna

Starting point: what is still open, and what was done

In a RAG system [\[1\]](#), the splitter decides which pieces of text can be found at all; every later step builds on it.

STILL OPEN

- How larger, more powerful models would sway the results
- What the model itself adds, beyond clean sentence boundaries, is not isolated

WHAT I DO

- Built an LLM chunker on a small local model and published it as a library
- Compared it with LangChain splitters matched to the same chunk length
- Tested on which document structure LLM chunking helps most

Does a language model place better chunk boundaries than the simple chunking techniques and under which conditions?

01

Build

An open source LLM-based chunker as a Python library, on PyPI, with LangChain and LlamaIndex adapters

02

Compare

Against fixed-size, recursive and semantic splitters, on three different document types

03

Test fairly

Same chunk length for every splitter and exact search

Background: rule-based splitting



Before embedding and search, every RAG pipeline cuts its documents into chunks. A passage that is cut apart cannot be retrieved as a whole.

Fixed-size [\[6\]](#)

HOW IT WORKS

Counts characters (or tokens) and cuts after every N of them, optionally repeating the end of one chunk at the start of the next (overlap).

WHERE IT CUTS

...the board meets every six months and reports...

STRENGTH · BLIND SPOT

Fast and predictable, but blind to everything: it cuts through words, sentences and sections.

Recursive (LangChain default) [\[2, 5\]](#)

HOW IT WORKS

Tries separators in order: paragraph break, then line break, then space. It only moves on to the next one when a piece is still longer than the target size.

WHERE IT CUTS

...the end of a short paragraph. ¶ The next...

...one long paragraph: every six months...

STRENGTH · BLIND SPOT

Keeps short paragraphs whole, but does not read the text: a long paragraph, as a PDF often produces, is still cut at a space.

Background: semantic and LLM-based chunking

Semantic (embedding-based) [3, 4, 6]

HOW IT WORKS

Turns every sentence into a vector, for example with Sentence-BERT, and compares neighbouring sentences. A new chunk starts where the similarity drops. Max-Min compares each new sentence with the whole current chunk.

STRENGTH

Follows the content, not the layout.

LIMITS

One embedding per sentence; the threshold is hard to tune and chunk sizes vary widely. On real documents rarely better than recursive splitting [6].

LLM-based (LumberChunker) [5]

HOW IT WORKS

Numbers the paragraphs, sends a group of them to a large LLM and asks at which paragraph the topic shifts.

RESULT

Higher recall than recursive splitting on 100 narrative books: 77.9 against 74.4 % of the answers among the top 20 chunks.

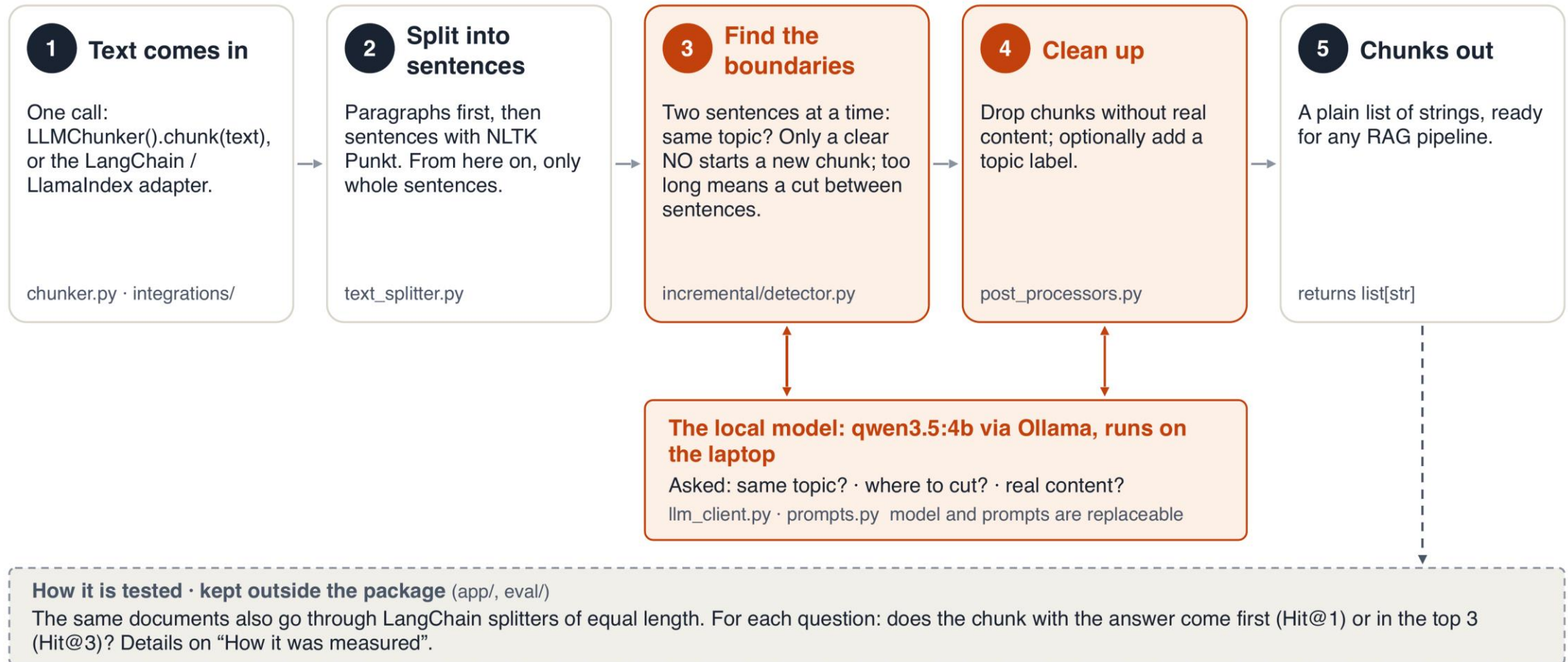
LIMITS

Slow and costly, 27 min instead of 0.6 s for one novel; relies on a large hosted model and on clean paragraphs.

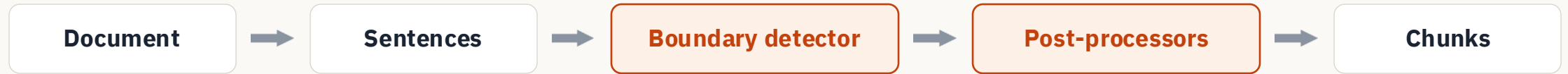
What this work changes

- Decides per sentence, not per paragraph: works even when a PDF loses its paragraphs, and never cuts inside a sentence.
- A small local model, qwen3.5:4b via Ollama: no API, repeatable with temperature 0 and a fixed seed.
- The model sees only the current chunk and the next two sentences: small calls, and the cost grows linearly with the document.
- A size cap that splits only between sentences; and optional post processes.
- Tested fairly: every splitter at the same chunk length

Project Architecture



How the splits are made



HOW THE SPLITS ARE MADE

LLM — call 1

Still the same topic?
semantic

LLM — call 2

Cap reached: where to cut?
size_cap

Pattern (+ LLM call variant)

A section heading
heading

LLM — calls 3 & 4

Low-info filter? · Enrichment
post-processes

How much of the chunking is really the model's decision?

Wells — prose

23 %

the size cap draws three of four boundaries

RFC 9110

47 %

the size cap draws the other half

NASA handbook

54 %

yet taking the model out makes no significant difference

Implementation: the core loop

```
while pos < len(sentences):
    start_idx = pos
    candidate = sentences[pos:pos + self.step_sentences]
    pos += self.step_sentences

    if not current:
        current = list(candidate)
    elif self._same_topic(current, candidate, start_idx):
        current.extend(candidate)
    else:
        # Natural topic boundary, close the chunk here
        chunks.append(" ".join(current))
        current = list(candidate)

    while self._over_limit(current):
        idx = self._fit_split(current)
        chunks.append(" ".join(current[:idx]))
        self.boundary_stats["size_cap"] += 1
        current = current[idx:]
```

The question the model gets

Current chunk: ...

Candidate sentences: ...

Do the candidate sentences continue the SAME topic? Answer YES or NO.

Only an explicit NO opens a new chunk.

From PyPI to chunks: two sections of RFC 9110

```
$ pip install llm-semantic-chunker  
$ ollama pull qwen3.5:4b
```

```
from llm_semantic_chunker import (  
    LLMChunker, OllamaClient)
```

```
chunks = LLMChunker(  
    client=OllamaClient(),  
    max_chunk_chars=1200  
)  
.chunk(text)
```

RESULT · RUN BEFOREHAND

5 chunks in 30.9 s

Input: 4,645 characters (sections 9.3.2 HEAD and 9.3.3 POST)

- [1] 9.3.2. HEAD The HEAD method is identical to GET ...
- [2] These minor inconsistencies are considered preferable ...
- [3] POST The POST method requests that the target resource ...
- [4] An origin server indicates response semantics by ...
- [5] Responses to POST requests are only cacheable when ...

How it was measured

THREE DOCUMENTS, ONE PER STRUCTURE TYPE

Document	Size	Questions
RFC 9110 [12] numbered sections	503k chars	549
NASA SE Handbook [13] PDF, 297 pages	814k chars	534
Wells, history [14] narrative prose	616k chars	400

One document per type as case studies

Questions

Written by an external LLM with pre-defined rules, Questions kept literal: each answer anchor is 1–2 sentences (≤ 300 chars), checked verbatim.

Hit@k

At least 80 % of the anchor, in one piece, inside one of the top k chunks. k = 1 and 3.

Retrieval

bge-m3 embeddings [\[11\]](#), exact cosine search instead of an approximate HNSW index [\[10\]](#)

Statistics

McNemar's exact test [\[8\]](#) on the paired results per question; Holm correction [\[9\]](#) over all 30 comparisons.

All splitters at the same chunk length

Hit@1 in %	NASA	RFC 9110	Wells
LLM chunker (this work)	69.5	74.9	84.0
Recursive, sentence ends first	66.9	71.6	89.8
Same chunker, model taken out	64.8	71.6	86.5
Semantic (embeddings), tuned	66.5	73.6	83.2
Recursive, LangChain default	61.6	71.8	87.2
Fixed-size	61.2	58.5	77.8

6 out of 30 significant after the Holm correction: RFC 9110 and NASA against fixed-size, NASA also against the LangChain recursive default, which are both splitters that cut sentences apart. Against the splitters that keep sentences intact, no significant lead.

Findings

Better boundaries than a rule?

Better than rules that cut sentences apart: significantly against fixed-size (RFC 9110, NASA) and LangChain default (NASA).

Not detectably better than rules that keep sentences intact, or than the same chunker without the model.

The optional features (heading detection, topic labels, the model-chosen split point) did not improve retrieval measurably.

IN PRACTICE

Length-matched splitting that respects sentence ends gets the same retrieval in about a second instead of hours; the model's gain does not justify its computing cost.

Lessons learned

WHAT WAS DIFFICULT

Evaluating fairly: building baselines that are truly comparable.

- Measuring significance, and separating what the LLM itself contributes
- Distilling a clear finding from the experiments
- Working within hardware limits, which made chunking very time-consuming

WHAT WAS EASY

Writing the first version of the chunker, and writing the thesis once the code was complete.

USE OF AI TOOLS

Claude Code in VS Code and browser-based AI tools: support with implementation and architecture, evaluation scripts and experiment runs, and tables and layout for the thesis.

References

- [1] Lewis, P. et al. (2020). [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#). NeurIPS 33.
- [2] Lavarec, E. & Du, Y. (2026). [Evaluating document chunking approaches for RAG in editorial content](#). IJ-AI 15.
- [3] Reimers, N. & Gurevych, I. (2019). [Sentence-BERT](#). EMNLP-IJCNLP.
- [4] Kiss, C., Nagy, M. & Szilágyi, P. (2025). [Max-Min semantic chunking of documents for RAG application](#). Discover Computing 28.
- [5] Duarte, A. V. et al. (2024). [LumberChunker: Long-Form Narrative Document Segmentation](#). Findings of EMNLP 2024.
- [6] Qu, R., Tu, R. & Bao, F. (2024). [Is Semantic Chunking Worth the Computational Cost?](#) arXiv:2410.13070.
- [7] Kiss, T. & Strunk, J. (2006). [Unsupervised Multilingual Sentence Boundary Detection](#). Computational Linguistics 32(4).
- [13] National Aeronautics and Space Administration (2016). [NASA Systems Engineering Handbook](#). NASA/SP-2016-6105 Rev2.
- pypi.org/project/llm-semantic-chunker
github.com/antunoviic/semantic-chunking
- [8] McNemar, Q. (1947). [Note on the Sampling Error of the Difference Between Correlated Proportions](#). Psychometrika 12(2).
- [9] Holm, S. (1979). [A Simple Sequentially Rejective Multiple Test Procedure](#). Scand. J. Statist. 6(2).
- [10] Malkov, Y. A. & Yashunin, D. A. (2020). [Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs](#). IEEE TPAMI 42(4).
- [11] Chen, J. et al. (2024). [BGE M3-Embedding](#). arXiv:2402.03216.
- [12] Fielding, R. T., Nottingham, M. & Reschke, J. (2022). [HTTP Semantics](#). RFC 9110.
- Software: Qwen 3.5 4B via Ollama; LangChain text splitters; Chroma; NLTK Punkt.
- [14] Wells, H. G. (1922). [A Short History of the World](#). The Macmillan Company; Project Gutenberg #35461.

Thank you for your attention

Questions are welcome.

<https://pypi.org/project/llm-semantic-chunker>

<https://github.com/antunoviic/semantic-chunking>